

智能网联汽车队列控制 – 任务开发指南

项目结构

```
project/
├── include/
│   ├── allocator.hpp           # 基于内存池的分配器（已完成）
│   ├── memory_pool.hpp        # 极致性能单线程内存池（已完成）
│   ├── timer.hpp              # 高精度计时器（已完成）
│   ├── brake_distance.h       # 任务A：间距调整（已完成）
│   └── ...                     # 其余9个任务的 .h 文件待添加
├── src/
│   ├── brake_distance.cpp      # 任务A的实现（已完成）
│   └── ...                     # 其余9个任务的 .cpp 文件待添加
├── Tasks.h                    # 任务调度入口（需扩展）
├── Tasks.cpp                  # main 函数（已完成）
└── README.md                  # 本文件
```

编码规范（必读）

1. 命名空间

- 所有代码必须位于 `Tasks` 命名空间内（除基础设施外）。
- 基础设施（内存池、分配器、计时器）已分别位于 `memory_pool`、`allocator`、`timer` 命名空间。

2. 类命名

- 每个任务对应一个类，类名采用 **PascalCase**，例如 `BrakeDistance`。
- 文件名与类名一致：`brake_distance.h` / `brake_distance.cpp`。

3. 成员变量命名

- 私有成员变量以下划线 `_` 开头，例如 `_pool`、`_result`、`_steps`。
- 公有成员函数使用小驼峰：`solve()`、`show_result()`。

4. 头文件保护

- 统一使用 `#pragma once`（现代 C++ 推荐）。

5. 注释

- 关键逻辑必须添加注释，尤其是算法思路。
- 使用 `//` 单行注释或 `/* */` 多行注释均可，保持清晰。



计时器使用指南

`timer::Timer` 是一个模板类，默认以毫秒为单位，可指定微秒、纳秒等。

```
#include "timer.hpp"

timer::Timer<timer::TimeUnit::Microseconds> timer; // 创建计时器（微秒级）
// Microseconds可替换为Seconds(秒)、Microseconds(微妙)、Nanoseconds(纳秒)
timer.start(); // 开始计时
// ... 执行任务 ...
timer.stop(); // 停止计时
timer.show_time(); // 直接打印耗时
```

注意：`show_time()` 会自动根据模板参数输出单位（如 `us`、`ms`）。

请在每个任务的 `taskX()` 函数中包裹需要计时的部分，参考 `taskA()` 的实现。



内存池与分配器

内存池 `memory_pool::MemoryPool<BlockSize>`

- 单线程、固定大小、无锁的内存池。
- 只能分配，不能回收（`deallocate` 为空操作）。
- 适合一次性分配大量小块内存的场景。

分配器 `allocator::PoolAllocator<T, BlockSize>`

- 包装了内存池，满足 STL 分配器接口。
- 可用于 `std::vector` 等容器，也可直接调用 `allocate()` / `deallocate()`。

使用示例（任务类内部）：

```
class MyTask
{
public:
    MyTask() : _pool(), _alloc(_pool), _buffer(nullptr) {}

    void solve()
    {
        _buffer = _alloc.allocate();           // 分配一个 char 大小的内存
        _alloc.construct(_buffer, 'X');        // 构造对象
        // 'X' 处为类型T的构造函数参数
        // 使用 _buffer...

        // 如果需要析构对象
        _alloc.destroy(_buffer);                // 析构对象

        // 如果在STL中使用
        std::vector<T, allocator::PoolAllocator<T, 64>> vec(_alloc);
    }

private:
    memory_pool::MemoryPool<64> _pool;          // 64字节池
    allocator::PoolAllocator<char, 64> _alloc;  // 分配器
    char* _buffer;
};
```

重要：

- 内存池大小 `BlockSize` 根据任务需求调整（通常 64~1024 字节足够）。
- `deallocate()` 和 `pool_.deallocate()` 均不做什么事，无需手动释放。

如何添加一个新任务（以任务B为例）

Step 1: 创建头文件 `include/taskx.h`

```

#pragma once
#include "allocator.hpp"
#include <print>

namespace Tasks
{
class TaskB
{
public:
    TaskB() : _pool(), _alloc(_pool), _result(nullptr) {}

    void solve(uint32_t input);        // 主求解函数
    void show_result() const;         // 打印结果

private:
    // 辅助函数 (如果有)
    void helper();

private:
    memory_pool::MemoryPool<128> _pool;        // 根据需求调整大小
    allocator::PoolAllocator<char, 128> _alloc; // 分配器
    // ... 其他成员

};

} // namespace Tasks

```

Step 2: 创建实现文件 `src/taskx.cpp`

```

#include "../include/taskx.h"
#include <print>

using namespace Tasks;

void TaskB::solve(uint32_t input)
{
    // 1. 分配结果缓冲区
    _result = _alloc.allocate();
    char* current = _result;

    // 2. 算法逻辑，构造结果字符串
    while (/* 条件 */) {
        *_current = 'X';           // 写入字符
        ++_current;
    }
    _steps = _current - _result;
    *_current = '\0';             // 字符串结束符

    // 3. 如果需要反转，调用 reverse_inplace (自行实现)
}

void TaskB::show_result() const
{
    std::print("{}\n{}\n", _steps, _result);
}

```

Step 3: 注册到 Tasks.h

在 Tasks.h 中添加头文件引用和 switch 分支：

```

#include "include/taskx.h"    // 添加
// ... 其他头文件 ...

class Tasks
{
public:
    void work(char choice)
    {
        switch (choice)
        {
            case 'A': taskA(); break;
            case 'B': taskB(); break;    // 新增
            // ... 后续任务
        }
    }
private:
    void taskA();    // 已在下方实现
    void taskB();    // 声明新任务
};

// 直接在类内实现
void taskB()
{
    timer::Timer<timer::TimeUnit::Microseconds> timer;
    uint32_t n;
    std::cin >> n;
    timer.start();
    TaskX tx;
    tx.solve(n);
    timer.stop();
    tx.show_result();
    timer.show_time();
}

```

Step 4: 无需修改 `Tasks.cpp`，main 函数自动适配。



通用注意事项

1. **输入输出格式**：严格遵循题目要求。
2. **数据类型**：数据类型禁止使用int，必须指明大小（如 `uint32_t`、`int64_t`），选用最贴合的范围
3. **不要使用全局变量**：所有状态封装在类内部。
4. **内存池大小选择**：
 - 如果结果字符串长度 ≤ 63 ，可用 `MemoryPool<64>`。
 - 如果长度 ≤ 127 ，用 `MemoryPool<128>`。
 - 如果长度不确定且较大，开足够大的内存池即可
5. **C++23 特性**：
 - 已启用 `std::print`（`<print>` 头文件），无需再包含 `<iostream>` 做格式化输出。
 - 如需输入，仍使用 `std::cin`。
6. **编码规范**：
 - 禁止在头文件中using namespace xxx，禁止在任何地方using namespace std。



任务列表（待完成）

标签	任务名称
A	BrakeDistance
B	
C	
D	
E	
F	
G	

H	
I	
J	

快速开始

1. 阅读 `brake_distance.h` 和 `brake_distance.cpp`，理解整体结构。
 2. 选择一个未完成的任务，按照上述步骤创建 `.h` 和 `.cpp`。
 3. 在 `Tasks.h` 中注册，编译运行测试。
 4. 使用 `timer` 测量性能，确保满足题目时限。
-